

Ce TP est constitué d'exercices dont le but est de pratiquer :

- la création de classes
- l'instanciation et la manipulation d'objets
- l'utilisation de git
- l'utilisation d'environnements virtuels

Exercice 1 : Arbres d'expression

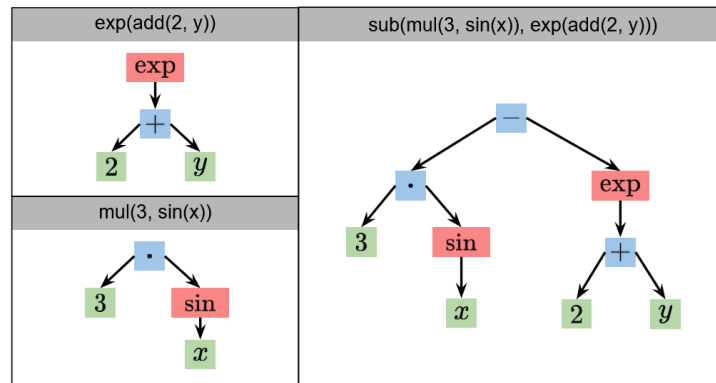


Figure 1 – Expressions mathématiques et arbres d'expression associés.

On souhaite écrire un programme permettant de manipuler des expressions mathématiques pouvant contenir des valeurs réelles, des variables, et des opérateurs, comme par exemple `mul(3, sin(x))`. Nous avons vu dans le TP précédent que l'on pouvait utiliser des arbres d'expression pour manipuler ces expressions mathématiques. En informatique, les arbres correspondent à un cas particulier de graphes, possédant une structure hiérarchique. Les noeuds tout en bas de la hiérarchie sont appelés "feuilles" de l'arbre, et le noeud le plus haut est appelé "racine" de l'arbre. Les arbres d'expression sont un cas particulier d'arbre dans lesquels les noeuds correspondent à des valeurs, des variables, ou des opérateurs. Trois exemples d'expressions mathématiques et arbres correspondants sont donnés dans la figure 1. Dans le premier exemple avec l'expression `exp(add(2, y))`, la racine de l'arbre d'expression est l'opérateur `exp`, et les deux feuilles de l'arbre sont la valeur 2 et la variable `y`. On peut également remarquer la distinction entre les opérateurs binaires (ayant deux branches filles) comme `+` `-` `*` représentés en bleu, et les opérateurs unaires (ayant une branche fille) comme `exp` et `sin`, représentés en rouge. Contrairement au TP 2, on va cette fois utiliser une syntaxe orientée objet pour définir et manipuler des arbres.

1. Dans un module `noeud.py`, définissez la classe `Noeud` ainsi que son constructeur. Un noeud est caractérisé par :
 - La valeur du noeud, par exemple "mul", "x", "sin", 2, 3.14. La valeur peut être de type `str` s'il s'agit d'une variable ou d'une opération, ou de type `int` ou `float` s'il s'agit d'une constante.
 - La liste des noeuds enfants, qui doit donc être une liste d'objets de type `Noeud`
2. Implémentez une méthode permettant d'ajouter un noeud à la liste des noeuds enfants.
3. Implémentez une méthode permettant d'afficher l'expression mathématique correspondant en utilisant l'affichage polonais vu au dernier TP.
4. Dans un nouveau fichier `main.py`, utilisez la classe `Noeud` pour instancier l'expression symbolique `exp(2+y)`. Vérifiez que son affichage avec la notation polonaise donne bien `exp + 2 y`.
5. Implémentez dans la classe `Noeud` la méthode `evaluer` qui permet d'évaluer l'expression symbolique pour une combinaison données des variables de l'expression. La méthode prend en argument un dictionnaire contenant les valeurs associées aux noms des variables, et retourne un nombre flottant. Si le dictionnaire ne donne pas la valeur associée à chaque variable, la méthode doit lever une `ValueError` avec un message adéquat. On supposera que les opérateurs possibles sont `+`, `-`, `*`. (Facultatif) Implémentez aussi les opérateurs `/`, `exp`, `log`, `sin`, `cos`.
6. Implémentez une méthode `tracer` qui utilise le package `matplotlib` pour tracer la fonction correspondant à votre expression symbolique. La méthode prendra en argument une chaîne de caractère correspondant à la variable qu'on souhaite tracer, et une liste de valeurs prises par cette variable. La méthode devra appeler la fonction `plot` du module `pyplot` de `matplotlib`, dont vous pouvez trouver un tutoriel ici : <https://matplotlib.org/stable/tutorials/pyplot.html>.

7. (Facultatif) Implémentez dans la classe `Noeud` la méthode `simplifiee` qui retourne un nouvel arbre d'expression en appliquant des simplifications à l'expression mathématique :
 - Les produits de 0 par X ou de X par 0 donne 0.
 - Les produits de 1 par X ou de X par 1 donnent X.
 - Le quotient 0/X donne 0.
 - Le quotient X/1 donne X.
 - La somme de X et 0 ou de 0 et X donnent X.
 - La soustraction de 0 à X donne X.
 - Si X ne comporte aucune variable, alors on peut le remplacer par la constante correspondant à son évaluation. Par exemple, remplacer $1 + \exp(0)$ par 2.
8. (Facultatif) Implémentez dans la classe `Noeud` la méthode `derivee` qui prend en argument une `str` indiquant par rapport à quelle variable on effectue la dérivation, et retourne un nouvel arbre d'expression correspondant à la dérivée.

Exercice 2 : Prise en main de git

Le but de cet exercice est de prendre en main git, vous pouvez vous appuyer sur le tutoriel disponible sur Moodle. Pour commencer l'exercice, vous devez avoir déjà créé un compte GitHub.

1. Sur votre ordinateur personnel, commencez par vous placer dans le répertoire utilisé pour l'exercice 1. Initialisez un dépôt local git en utilisant la commande `git init`. Ajoutez votre code de l'exercice 1 au prochain commit via la commande `git add <nom du fichier>`. Vous pouvez lister les fichiers ajoutés avec la commande `git status`. Normalement, vous devriez avoir deux modules Python à ajouter (le module `noeud.py` et le module `main.py`). Une fois les modifications ajoutées au prochain commit (affichées en vert), préparez le commit avec la commande `git commit -m "<votre message>"`.
2. Créez un dépôt (par exemple appelé TP4) sur votre compte GitHub. Liez votre dépôt local (sur votre ordinateur) avec le dépôt distant (sur GitHub) en utilisant la commande `git remote add origin <url de votre dépôt>`. Envoyez votre commit avec la commande `git push -u origin main`. Actualisez la page GitHub de votre dépôt pour vérifier que vos modifications ont bien été envoyées.
3. Pour cette question, vous allez devoir demander à un ou une collègue de vous ajouter comme collaborateur ou collaboratrice à son dépôt GitHub. Réciproquement, vous allez l'ajouter comme collaborateur ou collaboratrice à votre dépôt GitHub. Cela se fait en accédant aux paramètres de votre dépôt GitHub, dans l'onglet "collaborateurs", en cherchant votre collègue grâce à son nom d'utilisateur sur GitHub. Une fois que vous avez accès à son dépôt GitHub, placez-vous dans un nouveau dossier et importez son projet grâce à la commande `git clone <url de son dépôt>`. Vous devez normalement avoir ainsi récupéré ses fichiers, sur lesquels vous allez travailler dans la question suivante.
4. Changez de dossier pour vous placer dans le dossier que vous venez de cloner, et ouvrez ce-même dossier dans votre IDE. Ajoutez des docstrings à la classe `Noeud` du module `noeud.py`. Attention à documenter à la fois la classe et les méthodes (à l'exception du constructeur).
5. Ensuite, créez un nouveau commit contenant vos modifications sur le module `graph.py` du dépôt de votre collègue (commandes `git add` et `git commit`). Poussez ensuite vos modifications sur le dépôt distant de votre collègue (commande `git push`). Vérifiez sur GitHub que vos modifications ont bien été prises en compte.
6. Pour finir, ré-ouvrez dans VSCode votre projet du dernier TP, sur lequel votre collègue a poussé ses modifications. Ouvrez un terminal et lancez la commande `git pull` pour récupérer ses changements. Si tout fonctionne, vous devriez maintenant voir les docstrings qu'il ou elle a ajoutées.

Exercice 3 : Environnement

1. Ouvrez dans VSCode votre projet du dernier TP. On a vu au dernier TP que ce projet utilisait notamment deux packages : `networkx` et `matplotlib`, qui sont utilisées dans le module `utils.py` pour afficher les graphes. Dans cet exercice, vous allez créer un environnement virtuel avec `virtualenv` et y installer ces deux packages. Commencer par créer et activer un environnement virtuel :
 - (a) Ouvrez un terminal dans le dossier courant, et lancez la commande `virtualenv <nom_env>`. Vous pouvez prendre le nom que vous voulez pour votre environnement virtuel, par exemple "env_tp4". Si `virtualenv` n'est pas reconnu par votre machine, lancez la commande `pip install virtualenv` pour l'installer, puis réessayez la commande précédente pour créer un nouvel environnement virtuel.

- (b) Un nouveau dossier du nom de votre environnement a dû être créé dans votre répertoire. Ce dossier va contenir les packages permettant de lancer votre programme. Il faut maintenant activer l'environnement, pour que le terminal comprenne que vous souhaitez utiliser désormais la version de Python et les packages contenus dans l'environnement virtuel, et non pas ceux de votre machine. Pour activer l'environnement, lancez dans le terminal la commande suivante :

— Sur Windows (n'oubliez pas le point au début !) : `. \<nom_env>\Scripts\activate`

— Sur Linux et Mac : `source <nom_env>/bin/activate`

Une fois la commande exécutée, le nom de votre environnement devrait apparaître entre parenthèses dans le prompt du terminal. Cela signifie que l'environnement virtuel est activé. Tout ce que vous lancerez avec la commande `python` utilisera désormais la version de l'environnement virtuel.

2. Maintenant, nous allons pouvoir y installer les packages dont dépend le programme.

- (a) Essayez d'exécuter votre programme (`python main.py` ou `python3 main.py`). Normalement, vous devriez avoir une erreur qui vous indique que Python ne trouve pas de module qui s'appelle `matplotlib`. C'est normal : dans le nouvel environnement virtuel, nous n'avons pas encore installé `matplotlib`.
- (b) Créez un nouveau fichier texte `requirements.txt` dans votre répertoire. Dans ce fichier, écrivez, sur deux lignes `networkx` puis `matplotlib`, et sauvegardez le fichier.
- (c) Maintenant, lancez dans le terminal la commande `pip install -r requirements.txt`. Plusieurs packages vont être installés dans l'environnement virtuel : les deux packages demandés, auxquels il faut ajouter tous ceux dont ils dépendent. Cela peut prendre quelques temps.
- (d) Réessayez de lancer votre programme. Cela devrait normalement fonctionner.

3. Pour un gros projet, il peut parfois être fastidieux de récupérer à la main tous les packages qui sont utilisés. De plus, il peut être utile d'indiquer dans le fichier `requirements.txt` quelles sont les versions de chaque package avec lesquelles vous avez travaillé. Les packages Python évoluent au cours du temps, et certaines fonctionnalités que vous utilisez risquent d'être modifiées à l'avenir. Indiquer les versions dans le fichier `requirements.txt` permet de s'assurer qu'une personne voulant exécuter votre programme ait les bonnes versions de packages. Un outil pratique pour automatiser ce travail est `pipreqs`.

- (a) Pour commencer, on va installer `pipreqs`. Commencez par sortir de l'environnement, en exécutant dans le terminal la commande `deactivate`. Le nom de votre environnement qui était entre parenthèses dans le prompt du terminal devrait avoir disparu.
- (b) Maintenant que vous êtes sortis de l'environnement virtuel, installez `pipreqs` en exécutant dans le terminal la commande `pip install pipreqs`.
- (c) Une fois l'installation terminée, on va utiliser `pipreqs` pour automatiquement trouver les packages utilisés par votre programme, et les versions correspondantes. Lancez la commande suivante depuis le terminal : `pipreqs . --ignore <mon_env> --force`. L'option `--ignore <mon_env>` (en remplaçant par le nom de votre environnement) sert à indiquer à `pipreqs` que les packages installés dans l'environnement virtuel ne font pas partie du projet et qu'il ne faut pas les indiquer dans le fichier `requirements.txt`. L'option `--force` permet d'écraser le fichier `requirements.txt` existant. Vous pouvez maintenant réouvrir le fichier `requirements.txt`, il devrait contenir les deux mêmes package `matplotlib` et `networkx`, mais désormais avec la bonne version indiquée.

4. Pour finir, nous aimerions documenter notre projet sur la page Github, en indiquant comment installer et lancer notre projet pour les personnes qui voudraient l'utiliser.

- (a) Dans le répertoire courant, ajoutez un nouveau fichier texte `README.md`. L'extension `.md` indique qu'il s'agit d'un fichier `markdown`. Le langage `markdown` donne une syntaxe très simple pour écrire et mettre en page des documents. Il est typiquement utilisé dans les blogs, wikis, et documentations Github.
- (b) Sur moodle, vous pourrez trouver un exemple de fichier `README.md` à compléter pour ce TP. Sur VSCode, vous pouvez ouvrir une prévisualisation de ce à quoi ressemblera votre document. Une fois le fichier `README.md` ouvert dans VSCode, appuyez sur `Ctrl + Maj + V`, ou sinon, faites un clic droit sur l'onglet correspondant, puis dans le menu déroulant cliquez sur "Open Preview".

5. Dans cet exercice, vous avez normalement ajouté un fichier `README.md` et un fichier `requirements.txt` à votre projet. Ajoutez ces nouveaux fichiers à votre dépôt Github (commit, puis push). Attention ! Il ne faut surtout pas ajouter le dossier qui contient l'environnement virtuel à votre dépôt Github !